

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Debugging & Optimization Task

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 2 – Debugging & Optimisation Task
Weightage:	35% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	A.Likithasri	Reg. No.:	192511003
Section / Batch:	1	Date:	22/07/2026

Code of Conduct

I A.Likithasri(192511003)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Likitha.A

Instructions

- This test contains 20 Debugging & Optimisation Task questions. Total: 100 Marks (20*5).
- When a student answer using Scenario-Based, in evaluation the answer should be with following five requirements: Identify the problem type and constraints, Select appropriate data structure, Design the Solution, Optimization and efficiency, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Search Data Structure, Stack	Binary Search, Stack Operations	1 - 5	25
Linked List Data Structure	List Operations	6 – 10	25
Queue Data Structures	Queue Operations	11 - 15	25
Queue Data Structures, Stack Notations	Queue Operations, Expression Evaluation	16 - 20	25
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Debugging Accuracy	20	All errors corrected; code runs flawlessly	Most errors corrected; minor issues remain	Partial correction; code runs with issues	Code fails or major errors persist
Error Identification	30	Accurately identifies all logical, syntax, and edge-case errors	Identifies most major errors	Identifies some errors but misses key issues	Struggles to identify errors
Code Quality & Readability	20	Clean, modular, well-documented, follows best practices	Mostly clean and readable	Some structure but inconsistent	Poorly written, hard to understand
Explanation & Justification	20	Clear, logical, and well-justified reasoning with complexity analysis	Good explanation with some justification	Limited explanation; lacks depth	No or incorrect explanation
Testing & Validation	10	Comprehensive test cases including edge cases	Adequate test cases	Minimal testing	No testing provided

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q2
Stack
5 marksQ3
Stack
5 marksQ4
Stack
5 marksQ5
Stack
5 marksQ6
Linked List
5 marksQ7
Stack
5 marksQ8
Linked List Data Structures
5 marksQ9
Linked List
5 marksQ10
Queue
5 marks

10/10 answered

```
int binarySearch(int arr[], int n, int key) {  
    int low = 0, high = n - 1;  
    while(low < high) {  
        int mid = (low + high) / 2;  
        if(arr[mid] == key)  
            return mid;  
        else if(arr[mid] < key)  
            low = mid;  
        else  
            high = mid;  
    }  
    return -1;  
}
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 // Corrected Binary Search function  
4 int binarySearch(int arr[], int n, int key) {  
5     int low = 0;  
6     int high = n - 1;  
7  
8     // 1. Corrected Loop Condition: Include equality so the last element is checked  
9     while(low <= high) {  
10        int mid = low + (high - low) / 2; // Prevents potential integer overflow  
11  
12        if(arr[mid] == key) {  
13            return mid; // Target found  
14        }  
15        // 2. Corrected Boundary Shift: Step completely past the evaluated element  
16        else if(arr[mid] < key) {  
17            low = mid + 1;  
18        }  
19        else {  
20            high = mid - 1;  
21        }  
22    }  
23    return -1; // Target not present  
24 }  
25  
26 int main() {  
27     // Input Array must be sorted in ascending order  
28     int sortedArray[] = {11, 22, 33, 44, 55, 66};  
29     int size = sizeof(sortedArray) / sizeof(sortedArray[0]);  
30  
31     int searchKey;  
32     // Reads a single, separated raw search value from the console or file  
33     if (scanf("%d", &searchKey) == 1) {  
34         int outcomeIndex = binarySearch(sortedArray, size, searchKey);  
35  
36         if (outcomeIndex != -1) {  
37             printf("Element found at index %d\n", outcomeIndex);  
38         } else {  
39             printf("Element not found\n");  
40         }  
41     }  
42 }
```

Test Input

66

Output

Element found at index 5

⏱ Pending manual review

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

[← Back](#)

QUESTIONS

Q1
Binary Search
5 marks**Q2**
Stack
5 marks**Q3**
Stack
5 marks**Q4**
Stack
5 marks**Q5**
Stack
5 marks**Q6**
Linked List
5 marks**Q7**
Stack
5 marks**Q8**
Linked List Data Structures
5 marks**Q9**
Linked List
5 marks**Q10**
Queue
5 marks

10/10 answered

Q2 Stack

5 marks

Find the error in the stack deletion operation. Justify it.

```
int pop() {  
    if(top == 0) {  
        printf("Underflow");  
        return -1;  
    }  
    return stack[top--];  
}
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 #define MAX 5  
4  
5 int stack[MAX];  
6 int top = -1; // -1 indicates an empty stack  
7  
8 // Corrected pop function  
9 int pop() {  
10     // 1. Corrected Underflow Check for a 0-indexed array  
11     if (top == -1) {  
12         printf("Underflow\n");  
13         return -1;  
14     }  
15  
16     // 2. Return the top element, then post-decrement the top index  
17     return stack[top--];  
18 }  
19  
20 // Helper function to insert data  
21 void push(int data) {  
22     if (top == MAX - 1) return;  
23     stack[++top] = data;  
24 }  
25  
26 int main() {  
27     // Populate the stack with exactly one element to test the boundary  
28     push(99);  
29  
30     int trigger;  
31     // Reads a single, separated raw input from the console or file  
32     if (scanf("%d", &trigger) == 1) {  
33         int poppedValue = pop();  
34         if (poppedValue != -1) {  
35             printf("Successfully popped: %d (New top index: %d)\n", poppedValue, top);  
36         }  
37     }  
38  
39     return 0;  
40 }
```

Test Input

1

Output

Successfully popped: 99 (New top
index: -1)

⏸ Pending manual review

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

← Back

QUESTIONS

Q1

Binary Search
5 marks

Q2

Stack
5 marks

Q3

Stack
5 marks

Q4

Stack
5 marks

Q5

Stack
5 marks

Q6

Linked List
5 marks

Q7

Stack
5 marks

Q8

Linked List Data Structures
5 marks

Q9

Linked List
5 marks

Q10

Queue
5 marks

10/10 answered

Q3 Stack

5 marks

Check the snippet and identify the logical error.

```
int stack[5];  
int top;  
void init() {  
    top = 0;  
}
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 #define MAX 5  
4  
5 int stack[MAX];  
6 int top;  
7  
8 void init() {  
9     top = -1;  
10 }  
11  
12 void push(int data) {  
13     if (top == MAX - 1) {  
14         return;  
15     }  
16     top++;  
17     stack[top] = data;  
18 }  
19  
20 int main() {  
21     init();  
22     int userInput;  
23     if (scanf("%d", &userInput) == 1) {  
24         push(userInput);  
25     }  
26     return 0;  
27 }
```

Test Input

55

Output

⏱ Pending manual review

✔ Submitted

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

← Back

QUESTIONS

Q1
Binary Search
5 marksQ2
Stack
5 marksQ3
Stack
5 marksQ4
Stack
5 marksQ5
Stack
5 marksQ6
Linked List
5 marksQ7
Stack
5 marksQ8
Linked List Data Structures
5 marksQ9
Linked List
5 marksQ10
Queue
5 marks

10/10 answered

Q4 Stack

5 marks

Identify the errors in the following snippet.

```
if(ch == ')') {  
    while(stack[top] != ')') {  
        postfix[j++] = pop();  
    }  
    pop();  
}
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2 #include <stdlib.h>  
3  
4 #define MAX 20  
5  
6 char stack[MAX];  
7 int top = -1;  
8  
9 void push(char item) {  
10     if (top >= MAX - 1) return;  
11     stack[++top] = item;  
12 }  
13  
14 char pop() {  
15     if (top == -1) return '\0';  
16     return stack[top--];  
17 }  
18  
19 int main() {  
20     char postfix[MAX];  
21     int j = 0;  
22  
23     push('(');  
24     push('A');  
25     push('+');  
26     push('B');  
27  
28     char ch;  
29     if (scanf(" %c", &ch) == 1) {  
30         if (ch == ')') {  
31             while (top != -1 && stack[top] != '(') {  
32                 postfix[j++] = pop();  
33             }  
34             pop();  
35         }  
36         postfix[j] = '\0';  
37         printf("%s\n", postfix);  
38     }  
39     return 0;  
40 }
```

Test Input

```
)
```

Output

B+A

⌚ Pending manual review

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

← Back

QUESTIONS

Q1
Binary Search
5 marksQ2
Stack
5 marksQ3
Stack
5 marksQ4
Stack
5 marksQ5
Stack
5 marksQ6
Linked List
5 marksQ7
Stack
5 marksQ8
Linked List Data Structures
5 marksQ9
Linked List
5 marksQ10
Queue
5 marks

10/10 answered

Q5 Stack

5 marks

In balancing Bracket, you identify the error.

```
if((ch == '[' && stack[top] == '[') ||
(ch == ']' && stack[top] == '[')) {
    pop();
} else {
    return 0;
}
```

🐛 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 #define MAX 20
5
6 char stack[MAX];
7 int top = -1;
8
9 void push(char item) {
10     if (top >= MAX - 1) return;
11     stack[++top] = item;
12 }
13
14 char pop() {
15     if (top == -1) return '\0';
16     return stack[top--];
17 }
18
19 int checkBalanced(char ch) {
20     if (top != -1 && ((ch == '[' && stack[top] == '[') ||
21 (ch == ']' && stack[top] == '[') ||
22 (ch == '(' && stack[top] == '('))) {
23         pop();
24         return 1;
25     } else {
26         return 0;
27     }
28 }
29
30 int main() {
31     push('(');
32     push('[');
33
34     char ch;
35     if (scanf("%c", &ch) == 1) {
36         int result = checkBalanced(ch);
37         printf("%d\n", result);
38     }
39     return 0;
40 }
```

Test Input

]

Output

1

🔄 Pending manual review

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

← Back

QUESTIONS

Q1

Binary Search
5 marks

Q2

Stack
5 marks

Q3

Stack
5 marks

Q4

Stack
5 marks

Q5

Stack
5 marks

Q6

Linked List
5 marks

Q7

Stack
5 marks

Q8

Linked List Data Structures
5 marks

Q9

Linked List
5 marks

Q10

Queue
5 marks

10/10 answered

Q6 Linked List

5 marks

What is the issue?

```
temp = head
while temp.next != NULL:
    print(temp.data)
temp = temp.next
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct Node {
5     int data;
6     struct Node* next;
7 } Node;
8
9 Node* head = NULL;
10
11 void printList() {
12     Node* temp = head;
13     while (temp != NULL) {
14         printf("%d ", temp->data);
15         temp = temp->next;
16     }
17     printf("\n");
18     fflush(stdout);
19 }
20
21 void insertFront(int val) {
22     Node* newNode = (Node*)malloc(sizeof(Node));
23     if (newNode == NULL) return;
24     newNode->data = val;
25     newNode->next = head;
26     head = newNode;
27 }
28
29 int main() {
30     insertFront(30);
31     insertFront(20);
32     insertFront(10);
33
34     int trigger;
35     if (scanf("%d", &trigger) == 1) {
36         printList();
37     }
38     return 0;
39 }
```

Test Input

1

Output

10 20 30

Pending manual review

Submitted

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

[← Back](#)

QUESTIONS

Q1

Binary Search
5 marks

Q2

Stack
5 marks

Q3

Stack
5 marks

Q4

Stack
5 marks

Q5

Stack
5 marks

Q6

Linked List
5 marks

Q7

Stack
5 marks

Q8

Linked List Data Structures
5 marks

Q9

Linked List
5 marks

Q10

Queue
5 marks

10/10 answered

Q7 Stack

5 marks

Point out the error.
POP(stack):
if top == 0:
print("Underflow")
else:
return stack[top--]

★ Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX];
6 int top = -1;
7
8 int pop() {
9     if (top == -1) {
10         printf("Underflow\n");
11         return -1;
12     }
13     return stack[top--];
14 }
15
16 void push(int data) {
17     if (top == MAX - 1) return;
18     stack[++top] = data;
19 }
20
21 int main() {
22     push(10);
23     push(20);
24
25     int trigger;
26     if (scanf("%d", &trigger) == 1) {
27         int val = pop();
28         if (val != -1) {
29             printf("%d\n", val);
30         }
31     }
32     return 0;
33 }
```

Test Input

1

Output

20

Pending manual review

Submitted

QUESTIONS

Q1

Binary Search
5 marks

Q2

Stack
5 marks

Q3

Stack
5 marks

Q4

Stack
5 marks

Q5

Stack
5 marks

Q6

Linked List
5 marks

Q7

Stack
5 marks

Q8

Linked List Data Structures
5 marks

Q9

Linked List
5 marks

Q10

Queue
5 marks

10/10 answered

Q8 Linked List Data Structures

5 marks

Identify the errors in the following code

```
while current != NULL:
    current.next = prev
    prev = current
    current = current.next
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct Node {
5     int data;
6     struct Node* next;
7 } Node;
8
9 Node* head = NULL;
10
11 void reverseList() {
12     Node* prev = NULL;
13     Node* current = head;
14     Node* nextNode = NULL;
15
16     while (current != NULL) {
17         nextNode = current->next;
18         current->next = prev;
19         prev = current;
20         current = nextNode;
21     }
22     head = prev;
23 }
24
25 void insertFront(int val) {
26     Node* newNode = (Node*)malloc(sizeof(Node));
27     if (newNode == NULL) return;
28     newNode->data = val;
29     newNode->next = head;
30     head = newNode;
31 }
32
33 void printList() {
34     Node* temp = head;
35     while (temp != NULL) {
36         printf("%d -> ", temp->data);
37         temp = temp->next;
38     }
39     printf("NULL\n");
40 }
41
42 int main() {
43     insertFront(30);
44     insertFront(20);
45     insertFront(10);
46     printList();
47     reverseList();
48     printList();
49 }
```

Test Input

1

Output

30 -> 20 -> 10 -> NULL

🔴 Pending manual review

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

[← Back](#)

QUESTIONS

Q1

Binary Search
5 marks

Q2

Stack
5 marks

Q3

Stack
5 marks

Q4

Stack
5 marks

Q5

Stack
5 marks

Q6

Linked List
5 marks

Q7

Stack
5 marks

Q8

Linked List Data Structures
5 marks

Q9

Linked List
5 marks

Q10

Queue
5 marks

10/10 answered

Q9 Linked List

5 marks

What is the error in the below code?

new.next = head

head = new

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct Node {
5     int data;
6     struct Node* next;
7 } Node;
8
9 Node* head = NULL;
10
11 void insertFront(int val) {
12     Node* newNode = (Node*)malloc(sizeof(Node));
13     if (newNode == NULL) return;
14
15     newNode->data = val;
16     newNode->next = head;
17     head = newNode;
18 }
19
20 void printList() {
21     Node* temp = head;
22     while (temp != NULL) {
23         printf("%d -> ", temp->data);
24         temp = temp->next;
25     }
26     printf("NULL\n");
27 }
28
29 int main() {
30     int userInput;
31     if (scanf("%d", &userInput) == 1) {
32         insertFront(userInput);
33         printList();
34     }
35     return 0;
36 }
```

Test Input

42

Output

42 -> NULL

🕒 Pending manual review

📌 Submitted

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

[← Back](#)

QUESTIONS

Q1
Binary Search
5 marks**Q2**
Stack
5 marks**Q3**
Stack
5 marks**Q4**
Stack
5 marks**Q5**
Stack
5 marks**Q6**
Linked List
5 marks**Q7**
Stack
5 marks**Q8**
Linked List Data Structures
5 marks**Q9**
Linked List
5 marks**Q10**
Queue
5 marks

10/10 answered

Q10 Queue

5 marks

What is missing in the below pseudocode?

```
DEQUEUE(Q):  
if front == -1:  
    print("Underflow")  
else:  
    return Q[front++]
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 #define MAX 5  
4  
5 int queue[MAX];  
6 int front = -1;  
7 int rear = -1;  
8  
9 // Corrected DEQUEUE function  
10 int dequeue() {  
11     // 1. Check for Queue Underflow  
12     if (front == -1) {  
13         printf("Underflow\n");  
14         return -1;  
15     }  
16  
17     int data = queue[front];  
18  
19     // 2. Corrected Reset Logic: Check if this was the last element in  
20     if (front == rear) {  
21         front = -1;  
22         rear = -1;  
23     } else {  
24         // Otherwise, simply advance the front pointer  
25         front++;  
26     }  
27  
28     return data;  
29 }  
30  
31 // Helper function to insert data for testing  
32 void enqueue(int value) {  
33     if (rear == MAX - 1) return;  
34     if (front == -1) front = 0;  
35     queue[++rear] = value;  
36 }  
37  
38 int main() {  
39     // Populate the queue with exactly one item to test the boundary of  
40     enqueue(75);  
41  
42     int trigger;  
43 }
```

Test Input

1

Output

```
Successfully dequeued: 75 (Front  
reset to: -1, Rear reset to: -1)
```

Pending manual review

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS

Q1
Stack Notations - Postfix Expression
5 marks**Q2**
Circular Queue
5 marks**Q3**
Binary Search
5 marks**Q4**
Queue
5 marks**Q5**
Queue
5 marks**Q6**
Doubly Linked List
5 marks**Q7**
Stack
5 marks**Q8**
Stack
5 marks**Q9**
Circular Queue
5 marks**Q10**
Stack
5 marks

10/10 answered

Q1 Stack Notations - Postfix Expression

5 marks

Find the bug in the evaluation of a Postfix expression.

```
while (expression[i] != '\0') {  
    if (isdigit(expression[i])) push(expression[i] - '0');  
    else {  
        int op1 = pop();  
        int op2 = pop();  
        // perform operation: result = op1 + op2;  
        push(result);  
    }  
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2 #include <ctype.h>  
3  
4 #define MAX 100  
5  
6 int stack[MAX];  
7 int top = -1;  
8  
9 void push(int value) {  
10     stack[++top] = value;  
11 }  
12  
13 int pop() {  
14     return stack[top--];  
15 }  
16  
17 int evaluatePostfix(char expression[]) {  
18     int i = 0;  
19     while (expression[i] != '\0') {  
20         if (isdigit(expression[i])) {  
21             push(expression[i] - '0');  
22         } else {  
23             // The first pop is the right-hand operand  
24             int op1 = pop();  
25             // The second pop is the left-hand operand  
26             int op2 = pop();  
27             int result;  
28  
29             switch (expression[i]) {  
30                 case '+': result = op2 + op1; break;  
31                 case '-': result = op2 - op1; break;  
32                 case '*': result = op2 * op1; break;  
33                 case '/': result = op2 / op1; break;  
34                 default: result = 0; break;  
35             }  
36             push(result);  
37         }  
38         i++;  
39     }  
40     return stack[top];  
41 }
```

Test Input

23+

Output

Enter postfix expression: Result =
5

Pending manual review

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

[← Back](#)

QUESTIONS

- Q1**
Stack Notations - Postfix Expression
5 marks
- Q2**
Circular Queue
5 marks
- Q3**
Binary Search
5 marks
- Q4**
Queue
5 marks
- Q5**
Queue
5 marks
- Q6**
Doubly Linked List
5 marks
- Q7**
Stack
5 marks
- Q8**
Stack
5 marks
- Q9**
Circular Queue
5 marks
- Q10**
Stack
5 marks

10/10 answered

Q2 Circular Queue

5 marks

Identify the bug in this Circular Queue dequeue logic.

```
int dequeue() {  
    if (front == -1) return -1;  
    int data = queue[front];  
    front = (front + 1) % MAX;  
    return data;  
}
```

★ Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 #define MAX 5 // Maximum capacity of the queue  
4  
5 int queue[MAX];  
6 int front = -1;  
7 int rear = -1;  
8  
9 // Corrected dequeue function  
10 int dequeue() {  
11     // Check if the queue is empty  
12     if (front == -1) {  
13         return -1;  
14     }  
15  
16     int data = queue[front];  
17  
18     // Check if the queue has only one element left  
19     if (front == rear) {  
20         front = -1;  
21         rear = -1;  
22     } else {  
23         front = (front + 1) % MAX;  
24     }  
25  
26     return data;  
27 }  
28  
29 int main() {  
30     // Your main execution code goes here  
31     return 0;  
32 }
```

Submitted

Test Input

10

Output

Pending manual review

[← Back](#)

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

QUESTIONS

Q1

Stack Notations - Postfix
Expression
5 marks

Q2

Circular Queue
5 marks

Q3

Binary Search
5 marks

Q4

Queue
5 marks

Q5

Queue
5 marks

Q6

Doubly Linked List
5 marks

Q7

Stack
5 marks

Q8

Stack
5 marks

Q9

Circular Queue
5 marks

Q10

Stack
5 marks

10/10 answered

Q3 Binary Search

5 marks

Identify the bug in this Recursive Binary Search.

```
int binarySearch(int arr[], int l, int r, int x) {  
    int mid = l + (r - l) / 2;  
    if (arr[mid] == x) return mid;  
    if (arr[mid] > x) return binarySearch(arr, l, mid, x);  
    return binarySearch(arr, mid, r, x);  
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 // Your corrected binary search function  
4 int binarySearch(int arr[], int l, int r, int x) {  
5     if (l > r) return -1;  
6  
7     int mid = l + (r - l) / 2;  
8  
9     if (arr[mid] == x) return mid;  
10    if (arr[mid] > x) return binarySearch(arr, l, mid - 1, x);  
11  
12    return binarySearch(arr, mid + 1, r, x);  
13 }  
14  
15 int main() {  
16     // 1. INPUT ARRAY (Must be sorted)  
17     int arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};  
18     int n = sizeof(arr) / sizeof(arr[0]); // Size of the array (10 elements)  
19  
20     // 2. TARGET INPUT  
21     int x = 23;  
22  
23     // 3. EXECUTE USING BOUNDARY INPUTS (l = 0, r = n - 1)  
24     int result = binarySearch(arr, 0, n - 1, x);  
25  
26     // Output the result  
27     if (result == -1) {  
28         printf("Element %d is not present in array\n", x);  
29     } else {  
30         printf("Element %d is present at index %d\n", x, result);  
31     }  
32  
33     return 0;  
34 }
```

Test Input

stdin input (optional)

Output

Element 23 is present at index 5

Pending manual review

Submitted

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS

Q1

Stack Notations - Postfix
Expression
5 marks

Q2

Circular Queue
5 marks

Q3

Binary Search
5 marks

Q4

Queue
5 marks

Q5

Queue
5 marks

Q6

Doubly Linked List
5 marks

Q7

Stack
5 marks

Q8

Stack
5 marks

Q9

Circular Queue
5 marks

Q10

Stack
5 marks

10/10 answered

Q4 Queue

5 marks

Find the error in this Deque insertFront operation.

```
void insertFront(int data) {  
    if (front == 0) front = MAX - 1;  
    else front = front - 1;  
    deque[front] = data;  
}
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 #define MAX 5  
4  
5 int deque[MAX];  
6 int front = -1;  
7 int rear = -1;  
8  
9 void insertFront(int data) {  
10     if ((front == 0 && rear == MAX - 1) || (front == rear + 1)) {  
11         printf("Queue Overflow\n");  
12         return;  
13     }  
14  
15     if (front == -1) {  
16         front = 0;  
17         rear = 0;  
18     }  
19     else if (front == 0) {  
20         front = MAX - 1;  
21     }  
22     else {  
23         front = front - 1;  
24     }  
25  
26     deque[front] = data;  
27     printf("Successfully inserted %d at front index %d\n", data, front);  
28 }  
29  
30 int main() {  
31     int userInput;  
32  
33     // Reads a single, separated raw input from the console or file  
34     if (scanf("%d", &userInput) == 1) {  
35         insertFront(userInput);  
36     } else {  
37         printf("Invalid input error.\n");  
38     }  
39  
40     return 0;  
41 }
```

Test Input

45

Output

Successfully inserted 45 at front
index 0

⏱ Pending manual review

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS

Q1

Stack Notations - Postfix
Expression
5 marks

Q2

Circular Queue
5 marks

Q3

Binary Search
5 marks

Q4

Queue
5 marks

Q5

Queue
5 marks

Q6

Doubly Linked List
5 marks

Q7

Stack
5 marks

Q8

Stack
5 marks

Q9

Circular Queue
5 marks

Q10

Stack
5 marks

10/10 answered

Q5 Queue

5 marks

Identify the error in the Queue enqueue logic.

```
void enqueue(int data) {  
    if (rear == MAX) return;  
    queue[rear++] = data;  
}
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 #define MAX 5  
4  
5 int queue[MAX];  
6 int front = -1;  
7 int rear = -1;  
8  
9 void enqueue(int data) {  
10     // 1. Corrected Overflow Check  
11     if (rear == MAX - 1) {  
12         printf("Queue Overflow\n");  
13         return;  
14     }  
15  
16     // 2. Corrected Front Pointer Initialization  
17     if (front == -1) {  
18         front = 0;  
19     }  
20  
21     // 3. Move rear up and insert data safely  
22     rear++;  
23     queue[rear] = data;  
24     printf("Successfully enqueued %d (Front index: %d, Rear index: %d)\n",  
25         data, front, rear);  
26 }  
27  
28 int main() {  
29     int userInput;  
30  
31     // Reads a single, separated raw input from the console or file  
32     if (scanf("%d", &userInput) == 1) {  
33         enqueue(userInput);  
34     } else {  
35         printf("Invalid input error.\n");  
36     }  
37     return 0;  
38 }
```

Test Input

85

Output

```
Successfully enqueued 85 (Front  
index: 0, Rear index: 0)
```

🕒 Pending manual review

📌 Submitted

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS

Q1
Stack Notations - Postfix Expression
5 marks**Q2**
Circular Queue
5 marks**Q3**
Binary Search
5 marks**Q4**
Queue
5 marks**Q5**
Queue
5 marks**Q6**
Doubly Linked List
5 marks**Q7**
Stack
5 marks**Q8**
Stack
5 marks**Q9**
Circular Queue
5 marks**Q10**
Stack
5 marks

10/10 answered

Q6 Doubly Linked List

5 marks

Identify the error in this Doubly Linked List node deletion.

```
void deleteNode(Node* del) {  
    if (del->prev != NULL)  
        del->prev->next = del->next;  
    if (del->next != NULL)  
        del->next->prev = del->prev;  
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2 #include <stdlib.h>  
3  
4 typedef struct Node {  
5     int data;  
6     struct Node* prev;  
7     struct Node* next;  
8 } Node;  
9  
10 // Global head pointer input tracker  
11 Node* head = NULL;  
12  
13 // Corrected deletion function  
14 void deleteNode(Node* del) {  
15     // Base case: check if node to be deleted is NULL  
16     if (del == NULL) {  
17         return;  
18     }  
19  
20     // 1. Corrected Head Check: If node to be deleted is the head node  
21     if (head == del) {  
22         head = del->next;  
23     }  
24  
25     // Change next only if node to be deleted is NOT the last node  
26     if (del->next != NULL) {  
27         del->next->prev = del->prev;  
28     }  
29  
30     // Change prev only if node to be deleted is NOT the first node  
31     if (del->prev != NULL) {  
32         del->prev->next = del->next;  
33     }  
34  
35     // 2. Corrected Memory Bug: Free the allocated heap space  
36     free(del);  
37     printf("Node deleted successfully.\n");  
38 }  
39  
40 // Helper function to insert a node at the front  
41 void insertFront(int data) {
```

Test Input

18

Output

Node deleted successfully.

Pending manual review

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS

Q1
Stack Notations - Postfix
Expression
5 marks**Q2**
Circular Queue
5 marks**Q3**
Binary Search
5 marks**Q4**
Queue
5 marks**Q5**
Queue
5 marks**Q6**
Doubly Linked List
5 marks**Q7**
Stack
5 marks**Q8**
Stack
5 marks**Q9**
Circular Queue
5 marks**Q10**
Stack
5 marks

10/10 answered

Q7 Stack

5 marks

Identify the issue in the below pseudocode.

POP():

if top == 0 → Underflow

✳ Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX];
6 int top = -1; // -1 means the stack is completely empty
7
8 // Corrected POP function
9 int pop() {
10     // 1. Corrected Underflow Check (0-indexed array condition)
11     if (top == -1) {
12         printf("Stack Underflow\n");
13         return -1;
14     }
15
16     // 2. Retrieve data and decrement the top pointer index
17     int data = stack[top];
18     top--;
19
20     return data;
21 }
22
23 // Helper function to push data into the stack
24 void push(int data) {
25     if (top == MAX - 1) return;
26     stack[++top] = data;
27 }
28
29 int main() {
30     // Pre-populate stack with data elements
31     push(10);
32     push(20);
33     push(30); // Stack state: [10, 20, 30] -> top is at index 2
34
35     int trigger;
36     // Reads a single, separated raw input from the console or file
37     if (scanf("%d", &trigger) == 1) {
38         int poppedValue = pop();
39         if (poppedValue != -1) {
40             printf("Successfully popped value: %d\n", poppedValue);
41         }
42     }
43
44     return 0;
45 }
```

Test Input

1

Output

Successfully popped value: 30

Pending manual review

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS

Q1
Stack Notations - Postfix
Expression
5 marks**Q2**
Circular Queue
5 marks**Q3**
Binary Search
5 marks**Q4**
Queue
5 marks**Q5**
Queue
5 marks**Q6**
Doubly Linked List
5 marks**Q7**
Stack
5 marks**Q8**
Stack
5 marks**Q9**
Circular Queue
5 marks**Q10**
Stack
5 marks

10/10 answered

Q8 Stack

5 marks

Point out the error.
if top == MAX:
Overflow

🔍 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 3 // Small size to test overflow quickly
4
5 int stack[MAX];
6 int top = -1;
7
8 // Corrected PUSH function showing the overflow fix
9 void push(int data) {
10     // 1. Corrected Overflow Check: Stop before hitting MAX boundary
11     if (top == MAX - 1) {
12         printf("Stack Overflow\n");
13         return;
14     }
15
16     // 2. Increment pointer safely and store data
17     top++;
18     stack[top] = data;
19     printf("Successfully pushed %d onto stack (Top index: %d)\n", data, top);
20 }
21
22 int main() {
23     // Fill the stack capacity completely (MAX = 3)
24     push(10);
25     push(20);
26     push(30);
27
28     int userInput;
29     // Reads a single, separated raw input from the console or file
30     if (scanf("%d", &userInput) == 1) {
31         // This execution attempt will trigger the corrected overflow
32         push(userInput);
33     }
34
35     return 0;
36 }
```

Test Input

49

Output

```
Successfully pushed 10 onto
stack (Top index: 0)
Successfully pushed 20 onto
stack (Top index: 1)
Successfully pushed 30 onto
stack (Top index: 2)
```

⏱ Pending manual review

📌 Submitted

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS

Q1

Stack Notations - Postfix
Expression
5 marks

Q2

Circular Queue
5 marks

Q3

Binary Search
5 marks

Q4

Queue
5 marks

Q5

Queue
5 marks

Q6

Doubly Linked List
5 marks

Q7

Stack
5 marks

Q8

Stack
5 marks

Q9

Circular Queue
5 marks

Q10

Stack
5 marks

10/10 answered

Q9 Circular Queue

5 marks

Identify the issue in the following snippet for Circular Queue in Traffic Signal.

```
rear = (rear + 1) % MAX;
queue[rear] = vehicle;
if(rear == front){
    printf("Queue Full");
}
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 4 // Maximum slots in the traffic signal queue
5
6 char queue[MAX][20]; // Array of strings to store vehicle names
7 int front = -1;
8 int rear = -1;
9
10 void enqueueVehicle(char vehicle[]) {
11     // 1. Corrected Position: Validate Overflow BEFORE updating rear or front
12     if ((rear + 1) % MAX == front) {
13         printf("Queue Full\n");
14         return;
15     }
16
17     // 2. Initialise front if it is the first vehicle entering the signal
18     if (front == -1) {
19         front = 0;
20     }
21
22     // 3. Advance pointer safely and insert data
23     rear = (rear + 1) % MAX;
24     strcpy(queue[rear], vehicle);
25
26     printf("Vehicle '%s' added to signal queue (Rear: %d)\n", vehicle, rear);
27 }
28
29 int main() {
30     // Pre-populate the circular queue to near-capacity
31     enqueueVehicle("Car");
32     enqueueVehicle("Bus");
33     enqueueVehicle("Bike"); // 3 slots filled
34
35     char inputVehicle[20];
36     // Reads a single string input from the console or file
37     if (scanf("%19s", inputVehicle) == 1) {
38         // This will test the corrected queue full boundary condition
39         enqueueVehicle(inputVehicle);
40     }
41
42     return 0;
43 }
```

Test Input

Ambulance

Output

```
Vehicle 'Car' added to signal
queue (Rear: 0)
Vehicle 'Bus' added to signal
queue (Rear: 1)
Vehicle 'Bike' added to signal
queue (Rear: 2)
```

⏸ Pending manual review

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

[← Back](#)

QUESTIONS

Q1

Stack Notations - Postfix
Expression
5 marks

Q2

Circular Queue
5 marks

Q3

Binary Search
5 marks

Q4

Queue
5 marks

Q5

Queue
5 marks

Q6

Doubly Linked List
5 marks

Q7

Stack
5 marks

Q8

Stack
5 marks

Q9

Circular Queue
5 marks

Q10

Stack
5 marks

10/10 answered

Q10 Stack

5 marks

Point out the error?

PEEK(stack):
return stack[top+1]**Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX];
6 int top = -1; // -1 means the stack is empty
7
8 // Corrected Peek function
9 int peek() {
10     // 1. Check for stack underflow (empty stack)
11     if (top == -1) {
12         printf("Stack Underflow! Stack is empty.\n");
13         return -1;
14     }
15
16     // 2. Corrected Index: Return the current topmost element
17     return stack[top];
18 }
19
20 // Helper function to push data into the stack
21 void push(int data) {
22     if (top == MAX - 1) {
23         printf("Stack Overflow\n");
24         return;
25     }
26     top++;
27     stack[top] = data;
28 }
29
30 int main() {
31     // Populate the stack with some sample values
32     push(10);
33     push(20);
34     push(30); // Top element is now 30
35
36     int dummyInput;
37     // Reads a single, separated raw input to trigger the peek command
38     if (scanf("%d", &dummyInput) == 1) {
39         int topElement = peek();
40         if (topElement != -1) {
41             printf("Top element is: %d\n", topElement);
42         }
43     }
44
45     return 0;
46 }
```

Test Input

1

Output

Top element is: 30

 Pending manual review